



INT-LLPP: Lightweight in-band network-wide telemetry with low-latency and low-overhead path planning

Penghui Zhang^{a,*,}, Hua Zhang^a, Yuqi Dai^a, Cheng Zeng^a, Jingyu Wang^b, Jianxin Liao^b

^a National Mobile Communications Research Laboratory, Southeast University, Nanjing 211111, China

^b State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China

ARTICLE INFO

Keywords:

Network telemetry
Network measurement
Control overhead
Telemetry latency
Forwarding overhead

ABSTRACT

With the increasing complexity of networks, network telemetry becomes a critical part of network management. However, existing network telemetry systems still suffer from excessive control overhead, forwarding overhead, and latency.

In this paper, we propose INT-LLPP, a novel in-band network-wide telemetry system with low-latency and low-overhead path planning. The network telemetry architecture of INT-LLPP is unique in that it only requires a set of probes to collect telemetry items for multiple service flows. Moreover, the proposed Probe Path Generation (PPG) algorithm optimizes the probe paths to reduce the forwarding overhead and achieve full network coverage. To balance the telemetry latency and control overhead, we propose an efficient algorithm called the Simulated Annealing Maximum Latency Setting (SAMS) algorithm, which controls the length of the probe paths.

Simulation results show that INT-LLPP can reduce network telemetry control overhead by over 50% and reduce forwarding overhead by 5% to 10%. Moreover, INT-LLPP can lower telemetry latency by 30% to 40%.

1. Introduction

Nowadays, driven by advancements in technologies such as cloud computing [1], big data [2], and artificial intelligence [3], networks are experiencing unprecedented growth in scale [4]. At the same time, network services are becoming more diverse and complex. As a result, network administrators heavily rely on network telemetry technology to meet the strict requirements for bandwidth, latency, and stability in network maintenance and management. By providing real-time network status information, network telemetry plays an important role in various network management applications, such as traffic engineering [5,6], network security [7,8], and network fault diagnosis [9,10]. Network-wide telemetry can provide administrators with a complete and accurate view of the entire network [11], which enables better decision-making and timely responses to meet growing and changing network service requirements.

Despite the advantages of network telemetry, existing telemetry methods are still limited by high control overhead, forwarding overhead, and telemetry latency.

Control overhead: During network telemetry, control overhead refers to the bandwidth and control plane resources that are occupied by the control signals sent by the controller during network telemetry [12]. As the number of service flows in the network increases,

more probes are needed to collect network information. However, the generation, processing, and collection of these probes require a large number of control signals that may occupy storage space and cause network congestion. These factors can affect the transmission of normal packets [13].

Forwarding overhead: Network telemetry requires forwarding probes on the data plane, which consumes bandwidth and data plane resources. The forwarding overhead is used to describe the additional burden caused by forwarding probes. However, the network information collected by the probe increases [14], and the size of the probes can become very large. Forwarding these probes consumes bandwidth, leading to increased transmission latency and packet loss rates.

Telemetry latency: Telemetry latency is the time required for probes to be generated and received by the controller. Many network management applications require real-time telemetry information and high telemetry latency can cause network administrators to miss important events or delay decision-making. As a result, telemetry latency has become another critical factor [15,16] for tasks such as fault localization or security threat response.

At present, typical In-band Network Telemetry (INT) systems can be classified into Passive Network Telemetry (PNT) and Active Network Telemetry (ANT). To reduce the telemetry overhead, significant

* Corresponding author.

E-mail address: phzhang@seu.edu.cn (P. Zhang).

progress has been made in PNT research [17], such as PINT [14] and Sel-INT [18]. PNT relies on current network traffic, which cannot cover all network devices. However, it is essential to achieve full-network coverage to ensure that all network activities are monitored, including those occurring in less frequently accessed areas or devices. As a result, by adopting PNT it is difficult to completely cover the entire network and get a comprehensive overview of network behavior and performance, allowing for effective troubleshooting and optimization. Furthermore, the telemetry latency of PNT is determined by the packet transmission latency, which is difficult to actively control. Thus, telemetry latency is typically constrained by the inherent characteristics of the traffic, making real-time telemetry unachievable. In contrast, ANT systems are designed with special probes to collect network information actively [19]. However, as the network service flow increases, we require more probes to execute the telemetry task, which may cause an increase in control overhead and forwarding overhead. Additionally, the concurrent optimization of control overhead, forwarding overhead, and telemetry latency, which may lead to variable conflicts, remains a significant challenge.

In this paper, we proposed INT-LLPP, an ANT system specifically designed to achieve low control overhead, forwarding overhead, and telemetry latency. We propose a unique network telemetry architecture that integrates multiple types of telemetry tasks into the same set of probes, effectively reducing the number of probes needed. In addition, we propose the Probe Path Generation (PPG) algorithm to reduce forwarding overhead. Through this algorithm, the system can efficiently generate probe paths based on the service flows in the network. To reduce the number of control signals while ensuring the real-time nature of telemetry data, we also propose the Simulated Annealing Maximum Latency Setting (SAMLS) algorithm. In this way, the system can dynamically adjust the telemetry latency according to network service requirements, ensuring the timeliness and accuracy of telemetry information.

The main contributions of this paper are as follows.

- We introduce INT-LLPP, a novel network telemetry architecture. This architecture reduces the number of control signals by avoiding repeated detection of the same switch, which can effectively reduce control overhead.
- We propose the PPG algorithm, which plans the probe paths. The algorithm reduces the forwarding overhead while covering the whole network.
- We propose the SAMLS algorithm, a novel method for dynamically adjusting telemetry latency according to telemetry requirements. This approach balances control overhead and telemetry latency by adjusting the latency threshold.
- Through extensive simulations and testbed evaluations, we demonstrate that INT-LLPP improves telemetry performance compared to existing methods. Specifically, INT-LLPP reduces control overhead by over 50%, forwarding overhead by 5%, and telemetry latency by 30% to 40%.

The rest of this paper is organized as follows. In Section 2, we give a brief introduction to the related work. Then, we describe the architecture and critical designs of INT-LLPP in Section 3. Next, we build a mathematical model of probe path planning in Section 4. In Section 5, we describe the PPG algorithm and SAMLS algorithm. In Section 6, we simulated and discussed the performance of INT-LLPP. Finally, we summarize the paper in Section 7. The code and data are available at <https://github.com/785150084/INT-LLPP>.

2. Related work

In recent years, there has been some groundbreaking research in network telemetry systems. This section provides a brief overview of typical INT network systems, including PNT [14,18,20,21] and ANT [22–24] systems.

2.1. background

INT is a data collection framework that embeds telemetry data directly into data packets traversing the network. It provides real-time insights into network performance and behavior, enabling effective network management and optimization. INT is broadly categorized into two main architectures:

- **PNT:** This approach relies on leveraging existing network traffic to collect telemetry data without introducing additional control messages or probe packets. However, this passive methodology inherently limits the scope of telemetry coverage to the areas where sufficient traffic naturally exists, making it challenging to achieve comprehensive network visibility, especially in low-traffic or underutilized segments of the network.
- **ANT:** ANT actively injects dedicated probe packets into the network to collect telemetry data. These probes traverse predefined or dynamically calculated paths and gather detailed telemetry information at each network node they encounter. By relying on actively generated probes rather than existing traffic, ANT overcomes the limitations of PNT by achieving comprehensive network visibility, even in low-traffic or idle segments of the network.

This paper focuses on ANT systems, as they address the limitations of PNT by enabling full-network coverage and better control over telemetry latency. We discuss this in detail in Sections 2.2 and 2.3.

2.2. passive network telemetry

PNT passively collects telemetry data by embedding information into existing data packets [25], making the data packet itself the carrier of telemetry information. This approach offers numerous advantages, such as high precision, flexibility, and scalability [17].

However, despite its benefits, PNT is also subject to some limitations. Firstly, the telemetry coverage is restricted by service flows. If there are no service flows on certain network paths, PNT cannot obtain information about these paths, resulting in incomplete telemetry information [11]. Secondly, since PNT relies on current network traffic to collect network information, it cannot actively choose the forwarding paths and directly control telemetry latency. Finally, as collected telemetry information increases, the size of data packets increases, leading to additional forwarding overhead, and even packet truncation or loss [26].

To address the limitations, researchers have proposed various technologies, including PINT [14] and Sel-INT [18]. PINT limits the amount of information added to each data packet and disperses the information into multiple data packets to minimize the forwarding overhead of each data packet. However, PINT requires more data packets to complete a collection, resulting in higher control overhead. Sel-INT is another typical telemetry system that reduces control overhead by reducing telemetry frequency based on traffic conditions in the network [27]. However, this approach sacrifices telemetry accuracy or coverage. Besides, the telemetry latency of PNT is hard to reduce, as it depends on the current network flow. Therefore, it remains a challenge for these solutions to comprehensively address all the challenges of network telemetry.

2.3. Active network telemetry

ANT actively inserts the probe into the data plane and forwards it along the user-specified forwarding path [19]. The probe collects various network information [28], such as port delay and packet loss rate, during forwarding. Compared to PNT, ANT is more convenient for designing special schemes.

The two primary ANT systems are INT-path [23] and IntOpt [24]. INT-path generates probes and sends them to a target network to obtain

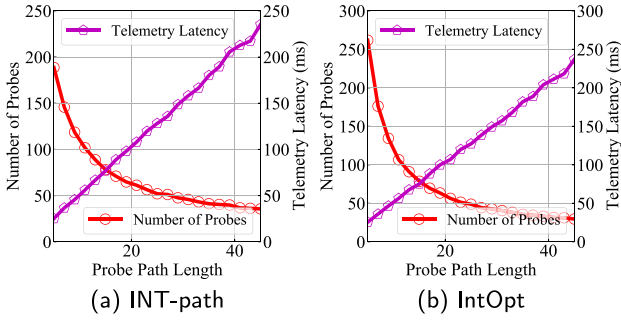


Fig. 1. Control overhead and telemetry latency of different telemetry systems.

network information. To reduce control overhead, INT-path designs the DFS algorithm and Eule Trail algorithm to deploy probe paths. However, INT-path does not address the problem of telemetry latency, and the deployed probe path length may be extremely long, which can impact the real-time performance of network information. In contrast, IntOpt monitors individual service flows by optimizing probe paths with a simulated annealing-based random greedy metaheuristic (SARG) algorithm, thereby reducing network overhead. Moreover, IntOpt sets a fixed latency limit to enhance telemetry's real-time performance.

Although ANT can achieve full network coverage, efforts are also made to reduce control overhead. However, in multiple service flow environments, the control overhead remains high due to the increased number of probes, making it difficult to extend to complex network systems. Therefore, further research is necessary. Additionally, telemetry latency in ANT is directly related to probe path length, presenting another problem to address. As shown in Fig. 1.a and Fig. 1.b, one way to regulate telemetry latency is by controlling the length of probe paths [11,29]. However, shorter probe paths result in greater control overhead since more probes are required to complete telemetry tasks [23]. Therefore, balancing telemetry latency and control overhead is a challenge.

3. System design of INT-LLPP

In this section, we describe the architecture and the critical designs of INT-LLPP. Then, we describe two specific design elements: the switch operations and probe packet format. Next, we introduce the method of telemetry results analysis.

3.1. Architecture of the INT-LLPP

INT-LLPP is committed to reducing the control overhead, forwarding overhead, and telemetry latency of the network telemetry system with full network coverage.

Technically, INT-LLPP uses the classic ANT architecture for network telemetry [23]. In common ANT systems, telemetry tasks are designed according to network services, with each network service having a corresponding service flow that outlines the path packets take from source to destination. Various telemetry tasks require the collection of diverse telemetry items, including latency, bandwidth, and more. As a result, different probe sets are required to perform telemetry tasks effectively. However, as the number of telemetry tasks increases, an explosion of probes in the network results in an increase in control overhead. This can negatively impact resource utilization and overall network performance. To reduce control overhead, the main idea of INT-LLPP is to use only one set of probes to collect all telemetry items of each programmable switch. In other words, INT-LLPP combines different telemetry tasks into one. This can significantly reduce the number of probes in the entire telemetry system, which also means less control overhead caused by probe generation and collection. In addition, as each switch adds different telemetry items to the probe,

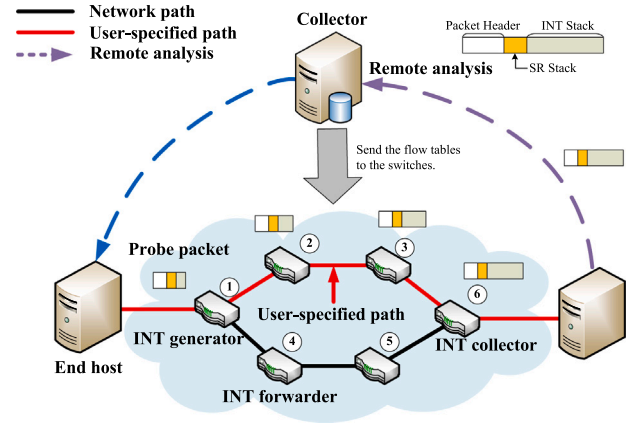


Fig. 2. Architecture of the INT-LLPP.

the labels in the same probe contain fields of different lengths. This feature of INT-LLPP introduced a new direction for the optimization of forwarding overhead, which will be discussed in Section 5. INT-LLPP has also designed a new probe path planning algorithm aimed at mitigating telemetry latency and reducing forwarding overhead.

Fig. 2 shows the architecture of the INT-LLPP. First, a mapping relationship is established between the telemetry items required by the service flow and the switches through which the service flow passes. In other words, each telemetry item is mapped to a switch that the service flow passes through so that the probes can get information about the corresponding telemetry items. Next, the controller sends or updates the flow tables to guide the programmable switches on how to identify, process, and forward the probes. It is worth noting that flow tables are an important component in programmable network devices, used to define how network packets are processed. We can control the behavior of programmable switches and implement various network functions by editing flow tables. The flow tables also define what telemetry items need to be added to the probes by each switch. Afterward, the switches process the packets according to the flow tables.

3.2. Switch operations

As shown in Fig. 2, the INT-LLPP contains the INT generators, the INT forwarders, and the INT collectors. The specific steps of the INT-LLPP routing mechanism are described as follows. Initially, the INT generator generates probes according to the probe packet format. Then, the INT generator adds its telemetry items into the probe and forwards the probe to the next port. Next, the INT forwarder should check whether a packet is a probe. If not, the INT forwarder processes and forwards the packet normally. Otherwise, the INT forwarder checks the switch ID and adds its all telemetry items to the probe. Afterward, the INT forwarder forwards the probe to the next port according to the user-specified INT path. Finally, when the probe arrives at the last switch in the path, the INT collector forwards the packet to the control plane after adding its all telemetry items to the probe. The control plane begins remote analysis.

3.3. Probe packet format

When creating the probe packet format, we should consider INT-LLPP's purpose of collecting all telemetry items using a set of probes. As depicted in Fig. 3, the probe packet header includes the Ethernet header, IP header, and UDP header. Then, the probe contains two label stacks: (i) Source Routing (SR) label stack, and (ii) INT label stack. Specifically, the SR label stack is a fixed-length stack in the UDP packet. Each SR label (1 Byte) represents a hop on the user-specified path. The INT label stack is a variable-length stack. Each INT label

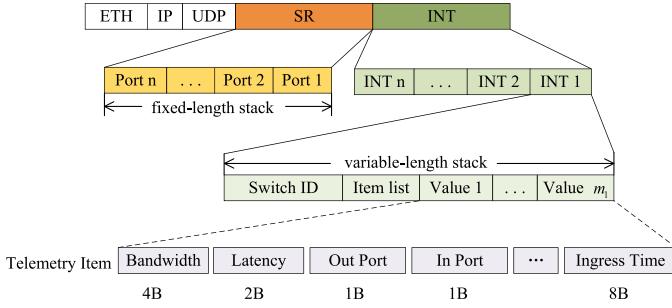


Fig. 3. Probe packet format.

contains “Switch ID”, “Item list”, and values. Specifically, the “Switch ID” represents the number of the switch. The “Item list” is used to mark the order of the telemetry items in each label stack to facilitate the controller to process the telemetry results. The “Value” represents the value of a telemetry item. Telemetry items include bandwidth, latency, out port, inport, ingress time, and so on. Different telemetry items have different field lengths that are labeled. For example, “4B” means a field length of four bytes.

3.4. Telemetry results analysis

After all probes are returned to the controller, each probe is parsed into a series of dictionaries. The network information collected from the switches is stored in a local database. The storage form is similar to $\{(switch\ ID, Type\ of\ a\ telemetry\ item): the\ value\ of\ a\ telemetry\ item\}$. Whenever a new probe returns, the controller replaces the corresponding information in the dictionary with the latest metadata. Administrators can query network status and obtain telemetry reports from the database. If the network information required by the administrators cannot be directly provided by the switches, such as path latency, path throughput, and so on, the controller can compute the network information from the relevant node information.

4. Network model and problem formulation

In this section, we first describe the network model of INT-LLPP, and then explain the problem definition of probe path planning.

4.1. Network model

In this subsection, we focus on network topologies where multiple network services are deployed and propose ways to map the telemetry requirements of each network service to the switches.

We consider a network topology consisting of n switches. Then, we define the network topology as an undirected physical graph, denoted by $G = (V, E)$. $V = \{i | i = 1, \dots, n\}$ is the set of physical nodes, where $i \in V$ represents the id of the switch. $E = \{e(i, j) | i, j \in V\}$ is the set of physical links between switches, where $e(i, j)$ represents the physical link between switch i and switch j . It is worth noting that $e(i, j) = e(j, i)$. If there is a connected physical link between switch i and switch j , then $e(i, j) \in E$ exists, otherwise, $e(i, j)$ does not exist.

We should map the telemetry items to each switch through the flow tables. Each switch combines to get all of his telemetry items. The set of telemetry items for switch i is represented as δ_i . The field length of the INT label for node i is affected by the set of telemetry items δ_i . The length of the INT label for switch i , which is represented as $b(i)$, includes the total length of all telemetry item fields in the set and the fixed-length “Switch ID” and “Item list” fields.

We use an example to illustrate the mapping process. As shown in Fig. 4, we consider a simple network composed of seven switches. This network has three different service flows, namely SF1, SF2, and SF3.

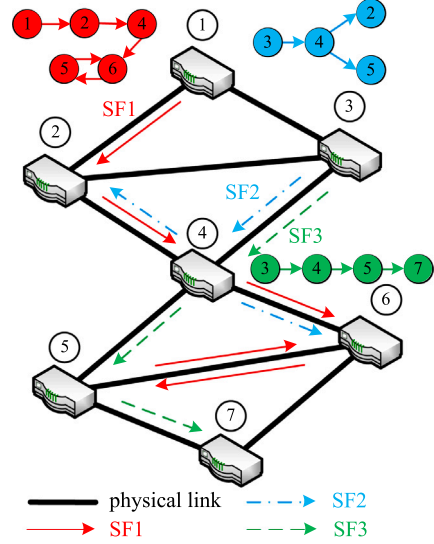


Fig. 4. Illustration of a network topology with three service flows.

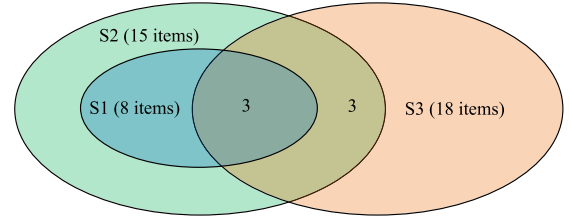


Fig. 5. Telemetry items required by service flows.

Table 1

Relative running time of the considered filters.

Node	Telemetry items	Node	Telemetry items
1	$S1$	5	$S1 \cup S3$
2	$S2$	6	$S2$
3	$S2 \cup S3$	7	$S3$
4	$S2 \cup S3$		

The telemetry items of each service flow are different. Let the sets of telemetry items required by SF1, SF2, and SF3 as sets $S1$, $S2$, and $S3$, respectively. The relationships between the telemetry items required by each service flow are shown in Fig. 5. Specifically, $S1$ has 8 telemetry items, $S2$ has 15 telemetry items, and $S3$ has 18 telemetry items. $S1$ is a subset of $S2$. $S3$ intersects with both $S1$ and $S2$, with 3 telemetry items shared between $S3$ and $S1$, and 6 telemetry items shared between $S3$ and $S2$. The set of telemetry items for each node is shown in Table 1. If a node has only one service flow passing through it, the set of telemetry items can be easily determined based on that service flow, such as nodes 1 and 7. If a node has multiple service flows passing through it, we need to merge the telemetry item sets of all the service flows as the set of telemetry items for that node. For example, for nodes 2 and 6, all telemetry items in the set $S1$ are covered by the set $S2$; thus $\delta_1 = \delta_6 = S2$. However, for nodes 3, 5, and 4, new sets of telemetry items need to be constructed to ensure that all telemetry items are collected.

4.2. Problem formulation

In this subsection, we build a mathematical model for the probe paths of the INT-LLPP system and analyze the system performance.

We first assume that the network telemetry system contains K probe paths. The k th probe path information is denoted by

$$f_k = [a_1^k, a_2^k, \dots, a_i^k, \dots, a_{N_k}^k], k = 1, \dots, K. \quad (1)$$

where $a_i^k \in V, i = 1, \dots, N_k$ represents the id of i th node which the k th probe passes. $N_k = |f_k|$ represents the number of nodes that the k th probe passing through. Because the length of the SR label stack is fixed, the k th probe flow's path length cannot exceed the capacity of the SR label stack. The number of bytes of each SR label is determined by the total number of ports in the network. In this paper, we define SR labels as 1 byte, which can represent 256 different ports and meet the port representation requirements of most networks. N_k should satisfy

$$N_k \leq N_{\max}, k = 1, \dots, K, \quad (2)$$

where $N_{\max}$ represents the number of bytes in the SR label stack, which once allocated will not be changed during telemetry.

The i th physical link in the k th probe is represented by $e(a_i^k, a_{i+1}^k)$. We denote the set of the k th probe flow's physical links by $L_k = \{e(a_i^k, a_{i+1}^k) | i = 1, \dots, N_k - 1\}$, $L_k \subseteq E$. After obtaining the set of links covered by each probe, we can denote the set of all probe paths' physical links L as

$$L = \bigcup_{i=1}^K L_i. \quad (3)$$

The probe path needs to cover all service flows to obtain accurate path-level information. As shown in Fig. 4, all links except links $e(1,3)$, $e(2,3)$, $e(5,7)$, and $e(6,7)$ should be covered by the probe paths. We represent the set of paths that all service flows pass through as L_s , $L_s \subseteq E$. Therefore, the set of physical links for all probe paths should be satisfied $L \supseteq L_s$.

Due to the link transmission capability and router processing capability [30], latency is a factor that cannot be ignored in the packet forwarding process. The latency information of each link can be obtained through the network telemetry of the previous cycle. Define the latency function $t: E \rightarrow T$. The latency for a packet to be forwarded from the node i to the node j is denoted by $t(i, j)$. If there is no connected physical link between nodes, $t(i, j)$ is infinite. According to the k th probe flow's information given by f_k , the latency of the k th probe flow per cycle is

$$T_k = \sum_{i=1}^{N_k-1} t(a_i^k, a_{i+1}^k). \quad (4)$$

where $t(a_i^k, a_{i+1}^k)$ is the latency of packet forwarding from node a_i^k to node a_{i+1}^k . Because we need to collect all probes before starting telemetry result analysis, the latency of network telemetry system T is equal to the highest of the probe path latency, which can be represented as

$$T = \max \{T_k\}, k = 1, 2, \dots, K. \quad (5)$$

The network consists of the data plane (also known as the forwarding plane) and the control plane. On the data plane, probes are forwarded between switches. This process incurs the forwarding overhead. The forwarding overhead c_f^k of the k th probe flow in the data plane can be expressed as

$$c_f^k = \sum_{j=1}^{N_k} \sum_{i=1}^j b(a_i^k) + N_k b_0, \forall k. \quad (6)$$

where b_0 is the field length of the packet header, $b(i)$ is the field length of the INT label that node i should add to the probe packet. Thus, $b(a_i^k)$ is the field length that i th node in the k th probe path should be added to the probe packet. The forwarding overhead of the network telemetry

system can be calculated as the sum of all probe flows' forwarding overhead, which is expressed as

$$C_f = \sum_{k=1}^K c_f^k. \quad (7)$$

On the control plane, the network telemetry also generates the control overhead. The control overhead is mainly caused by the generation and collection of probes. Therefore, the control overhead is related to the telemetry frequency and the number of probes [11,23,24], which is further determined by the probe paths' number K generated by the probe path planning algorithm. We represent control overhead c_c^k of the k th probe flow can be expressed as a linear function related to the number of paths, which can be expressed as

$$C_c = h \cdot K, \quad (8)$$

where h is the scaling factor [23,24].

The telemetry latency T and the control overhead C_c both are important considerations for network telemetry. To investigate the potential tradeoff between telemetry latency T and control overhead C_c , the weights of telemetry latency and control overhead, which are defined as ω_t and ω_c , are introduced to adjust different priorities for the two objectives. The telemetry cost of the telemetry system per cycle is defined as Q , which can be expressed as

$$Q = \omega_t \cdot T + \omega_c \cdot C_c \quad (9)$$

After mapping the service flows to the switches, we focus on the design of the probe path planning to minimize the telemetry cost Q and forwarding overhead C_f under the requirement of full network coverage. Specifically, we formulate the two optimization problems as follows:

$$\min_{f_k} C_f \quad (10)$$

$$\text{s.t. } L \supseteq L_s, \quad (10a)$$

$$N_k \leq N_{\max}, \forall k. \quad (10b)$$

and

$$\min_{f_k} Q \quad (11)$$

$$\text{s.t. } L \supseteq L_s, \quad (11a)$$

$$N_k \leq N_{\max}, \forall k. \quad (11b)$$

The constraint (10a) and constraint (11a) ensures that the probe paths cover the entire network topology. The constraint (10b) and constraint (11b) ensures that the path length does not exceed the capacity of the SR label stack.

Problem (10) and (11) are challenging to solve for the following reasons. On one hand, network topologies often consist of a large number of nodes and edges, requiring numerous combinations to be computed when searching for a solution. This results in a very high complexity of the problem and efficient algorithms are needed to obtain results in a reasonable time. On the other hand, since we need to consider latency, control overhead, and forwarding overhead at the same time, we should balance multiple objectives. This further increases the complexity of the problem. Therefore, solving problem (10) and (11) directly for the optimal solution is challenging. To overcome this, we propose the approximation algorithms in Section 5.

5. Proposed probe path planning algorithm

In this section, we propose the probe path planning algorithm that takes into account latency, control overhead, and forwarding overhead. The algorithm includes the PPG algorithm and the SAML algorithm. Specifically, the PPG algorithm generates probe paths based on the

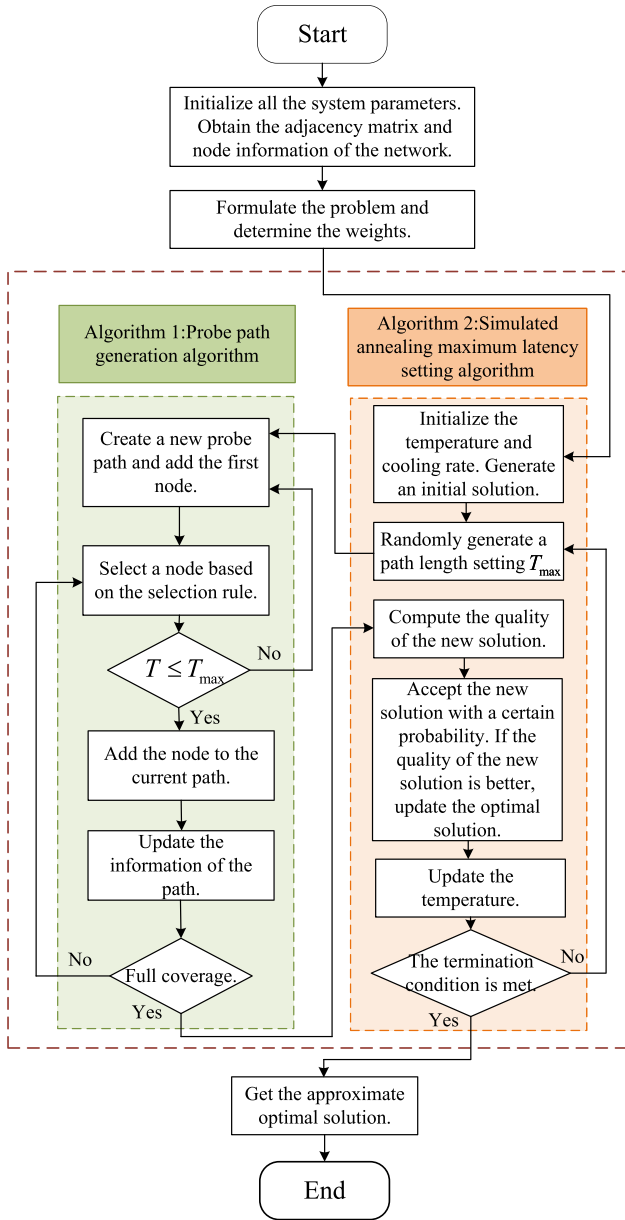


Fig. 6. Proposed probe path planning algorithm, including the PPG algorithm and the SAMLS algorithm.

telemetry items of each switch to achieve full network coverage while reducing forwarding overhead. The SAMLS algorithm is designed to determine a suitable maximum latency $T_{\max}$ for the PPG algorithm that balances telemetry latency and control overhead. Finally, we provide the complexity analysis for the proposed algorithms.

5.1. Probe path generation algorithm

In this subsection, we describe the PPG algorithm (Algorithm 1) for problem (10). As shown in Fig. 6, after creating a new probe path and adding the first node, we need to select a node to add to the probe path based on the selection rule. The selection rule is important to reduce forwarding overhead and satisfy the constraints of (10a) and (10b). Specifically, after adding any node, we should select a connected node and not all connected nodes are accessible. Assume that, when deploying the k th probe flow path $f_k, k = 1, 2, \dots, K$, we have added the node $a_{m-1}^k, m = 2, \dots, N_k$, and the k th path f_k can be tentatively

denoted as $f_k = [a_1^k, a_2^k, \dots, a_i^k, \dots, a_{m-1}^k], i = 1, \dots, m-1$. Then, the set $N(a_{m-1}^k)$ represents the nodes connected to the node a_{m-1}^k , which can be described as

$$N(a_{m-1}^k) = \{i | \exists e(a_{m-1}^k, i)\}, \quad (12)$$

where the symbol $\exists$ means the physical link $e(a_{m-1}^k, i)$ exists. When selecting node a_m^k , we should avoid unnecessary overhead caused by repeated probing of physical links. Thus, the physical link between the node a_m^k and the node a_{m-1}^k should not have been already covered, which can be expressed as

$$e(a_{m-1}^k, a_m^k) \notin L', \quad (13)$$

where $L' = \bigcup_{i=1}^k L_i$ is the set of links have been already covered. Next, we select node a_m^k from set $N(a_{m-1}^k)$ according to constraint (13). The node a_m^k requires inserting an INT label with field length $b(a_m^k)$ into the probe, and the probe also needs to be forwarded $N_k - m$ times to reach the INT collector. To reduce forwarding overhead, we formulate the selection rule for the node a_m^k as follows

$$a_m^k = \arg \min_i b(i), i \in N(a_{m-1}^k), \quad (14)$$

where the node a_m^k should subject to constraint (13). According to (6), we find that the earlier an INT label is added in a probe path, the more times it needs to be forwarded. The main purpose of the selection rule (14) is to avoid adding large INT labels to locations that require more forwarding times, which is valid to reduce the forwarding overhead of the probe path f_k . To facilitate understanding the selection rule (14), we implement path planning for a network with only three nodes as an example. For convenience, we denote the three nodes as i, j, k , and assume that the relationship between the INT label field sizes of the three nodes is $b(i) < b(j) < b(k)$. If the nodes join the path f_k according to the INT label field sizes from small to large, the path f_k can be represented as $f_k = [i, j, k]$. The forwarding overhead c_f^k for the path f_k is $c_f^k = 2b_i + b_j$. Compared with other path planning methods, the path f_k generated according to selection rule (14) brings minimal forwarding overhead. After the addition of node a_m^k , the path f_k should be updated with the information of the probe paths and start looking for the next node a_{m+1}^k in the same way. Note that, the first node a_1^k of the path f_k is a special case. We have a wider range of nodes to choose from and should select the most suitable node from the physical node set V . The selection rule is defined by

$$a_1^k = \arg \min_i b(i), i \in V, \quad (15)$$

where the node a_1^k should have a link that does not belong to set L' .

However, there are three cases where we end up adding nodes to the probe path f_k . In the first case, all links of the current node are already covered by L' . We cannot find any suitable node that satisfies constraint (13). In the second case, the latency of the probe path after adding the node a_m^k exceeds the maximum latency, i.e., $T_k + t(a_{m-1}^k, a_m^k) > T_{\max}$. We have to give up adding the node a_m^k . It should be noted that the maximum latency $T_{\max}$ is determined by the SAMLS algorithm (Algorithm 2), which is discussed in the next subsection. In the third case, when the number of nodes in the path reaches the maximum value $N_{\max}$, the SR label stack will not be able to add any more SR labels. We should complete the path planning of the path f_k and create a new path. In the above three cases, we should create a new path f_{k+1} and begin adding the first node a_1^{k+1} . Finally, to achieve network-wide visibility, we need to make sure that all the links in the network that need to be probed are covered, i.e., $L' \supseteq L_s$. After adding each node successfully, we should verify the coverage of the network telemetry system until the path planning is completed. To facilitate understanding and description, the proposed PPG algorithm is shown in Algorithm 1, and we describe the stepwise procedures as follows:

Algorithm 1 Probe Path Generation (PPG) Algorithm

```

1: Procedure PPG (matrix  $A$ ,  $T_{\max}$ )
2: while  $L' \subset L_s$  do
3:    $v \leftarrow 1$ ,  $t \leftarrow 0$ ,  $b \leftarrow b(1)$ ,  $n \leftarrow 1$ .
4:   for all  $i \in [1, \dots, n]$  do
5:     if  $b(i) < b$  then
6:        $v \leftarrow i$ ,  $b \leftarrow b(i)$ .
7:     end if
8:   end for
9:   Create a new path.  $flag \leftarrow 1$ .
10:  Add  $v$  into the current path.
11:  while  $flag = 1$  do
12:     $flag \leftarrow 0$ ,  $b \leftarrow 0$ ,  $v' \leftarrow 1$ 
13:    for all  $i \in N(v)$  do
14:      if  $b(i) < b$  and  $e(v, i) \notin L'$  then
15:         $v' \leftarrow i$ ,  $b \leftarrow b(i)$ ,  $flag \leftarrow 1$ .
16:      end if
17:    end for
18:     $t \leftarrow t + t(v, v')$ ,  $v \leftarrow v'$ ,  $n \leftarrow n + 1$ .
19:    if  $t < T_{\max}$  and  $flag = 1$  and  $n < N_{\max} + 1$  then
20:      Add  $v$  into the current path.
21:    else
22:       $flag = 0$ .
23:    end if
24:    Updated the information of  $L'$ .
25:    if  $L' \supseteq L_s$  then
26:      Break
27:    end if
28:  end while
29: end while

```

- (Step 1) *Initialization*: Initialize algorithm parameters and obtain network topology information through adjacency matrix A . For simplicity, the variable v represents the id of the selected node, t represents the current path latency, and b represents the field length of the INT label (line 3). Create the first path, and choose the first node according to selection rule (15) (lines 4–10).
- (Step 2) *Path construction*: Generate probe paths according to selection rule (14) (lines 13–17). Specifically, the link connected to the selected node should not be covered, and the INT label size of the node should be as small as possible.
- (Step 3) *Creation of new paths*: Set the global variable $flag$ to control the creation of new paths (line 12). Change the value of $flag$ to 0 (line 22) in the following three cases: (i) no node can satisfy (13), (ii) the path latency exceeds the maximum system latency $T_{\max}$, (iii) the number of nodes in the path reaches the maximum value $N_{\max}$. If $flag$ is 0, complete the current path and create a new path.
- (Step 4) *Termination test*: If the termination condition is satisfied, stop and complete all path planning tasks. Otherwise, update the path information, and continue to add nodes (line 2). Note that the termination condition could be that the network telemetry system covers the entire network topology or the maximum running time is reached (lines 25–27).

As shown in Fig. 7, we take the network which presented in Fig. 4 as an example to illustrate our proposed algorithm step by step. For simplicity, we assume that the labels' field lengths of each item in the case presented in Fig. 4 are consistent and that the latency for each path is the same. Additionally, we set a constraint on the telemetry latency, limiting it to no more than 4 hops. First, we traverse and find node 1, which adds the minimum amount of network information. We then create the first probe path, $path1 = \{1\}$. Among the nodes connected

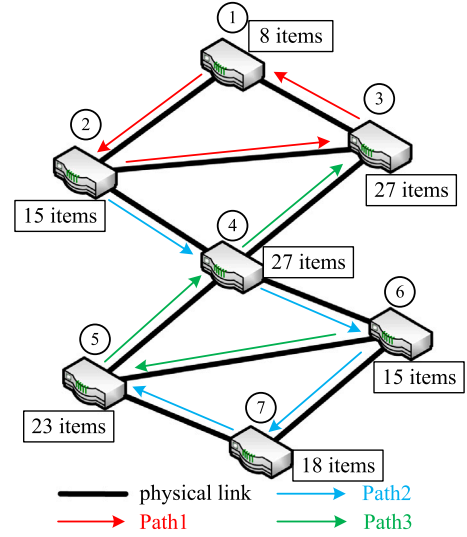


Fig. 7. Probe paths in the network.

to node 1, the telemetry item counts for nodes 2 and 3 are 15 and 27, respectively. Since node 2 has fewer telemetry items, node 2 is added to path1, i.e., $path1 = \{1, 2\}$. Next, we perform a local optimal selection for all nodes connected to node 2. Node 3, with the smallest telemetry item count among these, is added to path1, i.e., $path1 = \{1, 2, 3\}$. Similarly, node 4 is added to path1, i.e., $path1 = \{1, 2, 3, 4\}$. At this point, no other nodes connected to node 1 need to be probed, so path1 is completed. Next, we re-examine the topology to find any remaining unmonitored physical links and select node 2 as the starting point for path2, i.e., $path2 = \{2\}$. Using a similar process as for path1, we add nodes 4, 6, 7, and 5 to path2, i.e., $path2 = \{2, 4, 6, 7, 5\}$. At this point, if we were to add another node to path2, the telemetry latency would exceed the 4-hop limit. Thus, no further nodes can be added to path2. Once path2 is completed, we search for nodes that connect the remaining uncovered links and select node 7 as the starting point for path3, i.e., $path3 = \{7\}$. Following a similar process, we add nodes 5, 4, and 3 to path3, resulting in $path3 = \{7, 5, 4, 3\}$. Since the probes now cover the entire network, we plan the path with three non-overlapping probe paths (path1, path2, and path3).

5.2. Simulated annealing maximum latency setting algorithm

In this subsection, we describe the SAML algorithm (Algorithm 2), as shown in Fig. 6. The SAML algorithm is designed based on simulated annealing, which aims to select the approximate optimal solution with maximum latency $T_{\max}$. This scheme is inspired by solid-state annealing processes in physics and can help avoid the maximum latency generated randomly falling into local optima.

Algorithm 2 shows the details of the SAML algorithm, which consists of the following three steps.

- (Step 1) *Initialization*: Firstly, the temperature and cooling rate need to be initialized. Then, a random initial maximum latency $T_{\max}$ is generated as the starting point of the search, and the probe paths are generated through the PPG algorithm. The probe paths generated based on this initial maximum latency may be very poor, but it will be continuously improved with the increase of the number of iterations. (lines 2–4).

Algorithm 2 Simulated Annealing Maximum Latency Setting (SAMLS) Algorithm

```

1: Procedure SAMLS
2: Initialize the parameters.  $Q_p, Q_n, Q_b \leftarrow NULL$ .
3:  $temperature \leftarrow \lambda$ ,  $cooling \leftarrow \alpha$ .
4: Randomly generate an initial solution and get the current telemetry cost  $Q_p$ .
5: while  $temperature > 1$  do
6:   Set the maximum latency  $T_{max}$  randomly.
7:   Generate a new solution and get the new telemetry cost.  $Q_n \leftarrow PPG(A, T_{max})$ .
8:   if  $e^{(Q_n - Q_p)/\lambda} > Random(0, 1)$  then
9:     Update the current telemetry cost.  $Q_p \leftarrow Q_n$ .
10:    if  $Q_n > Q_b$  then
11:      Update the best telemetry cost.  $Q_b \leftarrow Q_n$ .
12:    end if
13:  end if
14:   $temperature \leftarrow temperature \times (1 - cooling)$ 
15: end while

```

(Step 2) *Iterative search*: In each iteration, a new maximum latency is generated as a new solution. If this new solution is better than the current one, it is directly accepted; otherwise, it is accepted with a certain probability. This probability is related to the temperature, and as the number of iterations increases, the temperature gradually decreases. Therefore, in the early stages, the probability of accepting inferior solutions is higher, while in the later stages, the probability of accepting inferior solutions becomes smaller and smaller. (lines 5–12).

(Step 3) *Cooling strategy*: During the iterative process, it is necessary to continuously reduce the temperature to control the probability of accepting inferior solutions. When the temperature is less than 1 or reaches the maximum number of iterations, the algorithm will terminate. (line 14).

5.3. Complexity analysis

The same problem can be solved by different algorithms, and the quality of an algorithm affects the efficiency of the program. If the algorithm requires an inordinate amount of resources (such as storage space and running time), it is still not worth promoting in practice. Thus, the computational complexity of an algorithm is critical to its implementation. Generally, the evaluation of an algorithm is mainly considered from the run-time complexity and space complexity [31]. Specifically, the space complexity describes the storage space temporarily occupied by an algorithm during its operation, and the run-time complexity qualitatively describes the time required for an algorithm. In this subsection, we analyze the computational complexity of the PPG algorithm and the SAMLS algorithm respectively.

5.3.1. Space complexity

The main data structures of the PPG algorithm include the adjacency matrix A , the set of covered paths L' , and some auxiliary variables such as v , t , $flag$, etc. The adjacency matrix A is a symmetric matrix that can be represented as

$$A = \begin{matrix} & \begin{matrix} v_1 & v_2 & \cdots & v_n \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{matrix} & \begin{pmatrix} e(1,1) & e(1,2) & \cdots & e(1,n) \\ e(2,1) & e(2,2) & \cdots & e(2,n) \\ \vdots & \vdots & \ddots & \vdots \\ e(n,1) & e(n,2) & \cdots & e(n,n) \end{pmatrix} \end{matrix} \quad (16)$$

The storage space it occupies can be represented as $O(n^2)$. The set of covered paths L' contains at most n^2 elements, so the storage space does not exceed $O(n^2)$. For other auxiliary variables, their space complexity

is very small and can be ignored. Therefore, the total space complexity of the PPG algorithm is $O(n^2)$.

Then, we analyze the space complexity of the SAMLS algorithm. The main data structures of the SAMLS algorithm include the adjacency matrix A , three variables used to store the objective function values Q_p, Q_n, Q_b , and some auxiliary variables such as t and c . The storage space occupied by the adjacency matrix A is $O(n^2)$, while the space complexity of the objective function value variables is $O(1)$. Other auxiliary variables can also be ignored. Therefore, the total space complexity of this algorithm is $O(n^2)$.

In summary, the space complexity of both algorithms is $O(n^2)$. Because existing telemetry schemes, such as SARG [24], DFS and Euler Trail [23], also need to store adjacency matrices, the space complexity of the proposed algorithms are not significantly improved. It should be noted that in practical applications, the space complexity of the algorithm may be influenced by factors such as hardware devices and program implementation environment and the specific space usage may vary.

5.3.2. Run-time complexity

We analyze the run-time complexity of the PPG algorithm. For simplicity, except for the operations related to the data processing, we exclude other operations, such as assignment. The time required to find a node depends on the adjacency matrix $A \in \mathbf{R}^n$, where the time to visit each node depends on the number of nodes, and the time required to find a connected node after visiting each node also depends on the number of nodes. Thus, the total time required to generate the paths is $O(n^2)$. Moreover, to ensure the entire network is covered, we have the requirement to additionally verify $L' \subset L_s$, and the extra time required is $O(|L_s|)$. Therefore, the run-time complexity of PPG algorithm is $O(|L_s| \cdot n^2)$. In the worst case, where L_s is equal to E , the time complexity is $O(|E| \cdot n^2)$.

Next, we combine the SAMLS algorithm with the PPG algorithm and analyze the overall run-time complexity. In the SAMLS algorithm, the total number of iterations, denoted as p , is related to temperature and cooling rate. When $\lambda \cdot \alpha^p \leq 1$, the iteration stops. It can be seen that in the worst case, the algorithm needs to perform p iterations, which is given by

$$p = \frac{\log(1/\lambda)}{\log \alpha}, \quad (17)$$

where λ is the initial temperature and α is the cooling rate. Each iteration requires executing an algorithm with a complexity of $O(|E| \cdot n^2)$. Therefore, the total time complexity is $O(|E| \cdot n^2 \cdot \log(1/\lambda)/\log(\alpha))$. Note that the time complexity of temperature adjustment is ignored here because it is usually small.

Our analysis shows that the time complexity of the proposed algorithms is higher compared to DFS and Euler Trail [23]. However, it is important to note that the proposed algorithms still demonstrate a relatively low time complexity when compared to other algorithms, such as SARG [24]. Considering the experimental results and practical requirements, the time complexity of the proposed algorithms can meet the needs of updating probe paths for network telemetry systems.

6. Experimental evaluations

6.1. simulation setup

In this subsection, we simulate the proposed algorithms. We first compared the performance of the INT-LLPP architecture with IntOpt [24], INT-path [23] and NetView [32]. Then, we analyzed the performance of the PPG and SAMLS algorithms by comparing them against three other algorithms: SARG [24], DFS, Euler Trail [23] and the probe path planning algorithm in NetView [32]. Specifically, we evaluated their efficiency in terms of forwarding overhead, control overhead, and telemetry latency. Next, we introduce the routing mechanism and probe path planning algorithms of existing network telemetry systems in detail.

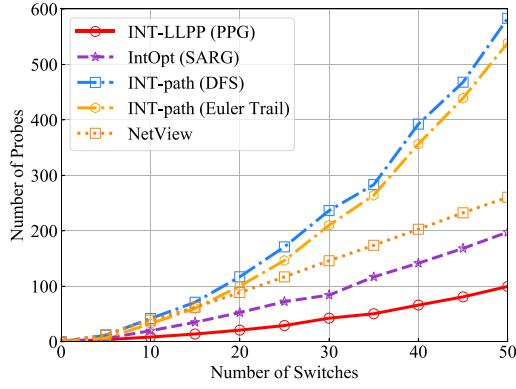


Fig. 8. Relationship between the number of probes and the number of switches with fixed telemetry latency threshold.

- INT-path [23]: This method allows probes to collect specific types of metadata using two algorithms for path planning: DFS and Euler Trail. DFS offers low complexity, while Euler Trail generates non-overlapping INT paths covering the entire network with fewer paths. However, neither algorithm addresses telemetry latency. In our simulations, a fixed latency limit is set for both algorithms.
- IntOpt [24]: IntOpt enables probes to collect multiple metadata types simultaneously, ensuring consistency in collected metadata. To reduce control overhead, it employs the SARG algorithm, based on simulated annealing, with a fixed latency limit to avoid excessively long probe paths.
- NetView [32]: NetView is designed for scenarios where only one switch generates and collects probes. Breaking down path-level data into node-level data, reduces telemetry coverage requirements, making it suitable for switch-constrained environments. However, the single-switch assumption limits its applicability in diverse networks.

To evaluate the performance of different algorithms, we create random topologies in which nodes are connected with a certain probability. Considering the clogged drain effect and the straggling herd effect [33], we refer to the prototype on Mininet and adopt the latency setting scheme proposed in [34]. Unless otherwise noted, the simulation parameters are set as follows. The field length of the packet header b_0 is 50 bit. Each service flow randomly collects one to six telemetry items, and the field length of each telemetry item is randomly 1B, 4B, or 8B. Considering the scale of the network topologies, the fixed telemetry latency threshold of the comparison algorithms is 25 ms [11]. Note that the probe path planning algorithm in NetView cannot set the telemetry latency threshold.

6.2. Evaluation results

In this subsection, we conducted several analyses to evaluate the performance of different frameworks and algorithms. We first investigated the impact of the INT-LLPP framework on control overhead by examining its efficacy in collecting telemetry items from multiple network devices. Then, we assessed the impact of the PPG algorithm on forwarding overhead. Finally, we conducted an in-depth analysis of the SAMS algorithm's influence on telemetry latency in different scenarios.

6.2.1. The impact of the INT-LLPP framework on control overhead

When analyzing control overhead, we need to eliminate the impact of telemetry latency. For fairness, we set a fixed maximum latency threshold for all network telemetry systems to control the length of

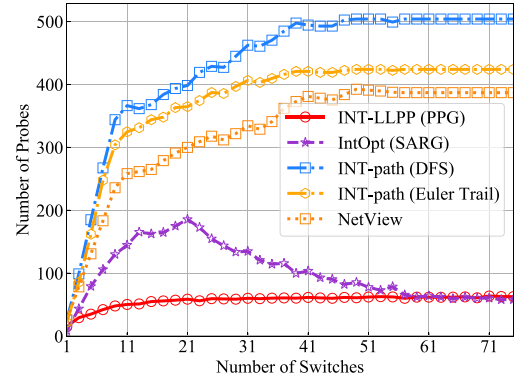


Fig. 9. Relationship between the number of probes and the number of network services with fixed telemetry latency threshold.

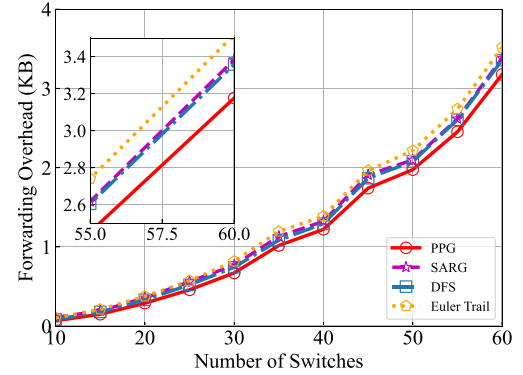


Fig. 10. Relationship between the forwarding overhead and the number of switches with fixed telemetry latency threshold.

probe paths. The control overhead is related to the number of probes. Fig. 8 shows the relationship between the number of probes and the number of switches with fixed telemetry latency threshold. We can find that the probe numbers of all four telemetry systems keep increasing with the number of switches. Specifically, compared with INT-path, NetView and IntOpt, INT-LLPP significantly reduces the number of probes per cycle. Since the probes in INT-LLPP have the best flexibility to collect different telemetry items, switches will not be redundantly probed by multiple probes.

Fig. 9 shows the relationship between the number of probes and the number of network services. As the number of network services increases, the number of probes in each algorithm also gradually increases and eventually stabilizes around a specific value. INT-LLPP requires the least number of probes. It is worth noting that the trend of the number of probes required by IntOpt first increases and then decreases. This is because the continuous increase of network services gradually requires almost all telemetry items to be provided by each switch. In this case, the number of probes required by IntOpt is very close to that of INT-LLPP.

6.2.2. The impact of the PPG algorithm on forwarding overhead

This section examines the impact of the PPG algorithm on forwarding overhead. Due to the complexity of NetView's calculation process, which involves splitting link-level indicators, a direct comparison is omitted for simplicity. Fig. 10 shows that the forwarding overhead generated by the PPG algorithm is the smallest. Compared with other algorithms, the forwarding overhead of the PPG algorithm has decreased by 5% to 10%. As shown in Fig. 11, in a network composed of 30 switches, the forwarding overhead generated by the four algorithms increases with the telemetry latency threshold. A larger

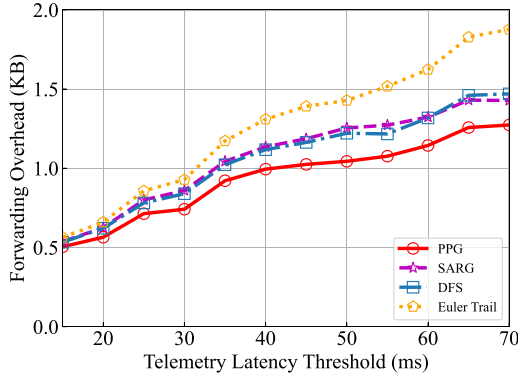


Fig. 11. Relationship between the forwarding overhead and telemetry latency threshold.

telemetry latency threshold means longer probe paths, which leads to larger probes and more forwarding times, ultimately increasing the forwarding overhead. We can see that as the telemetry latency threshold increases, the performance improvement of the PPG algorithm on forwarding overhead becomes more obvious compared with other algorithms. At the same time, the Euler Trail algorithm tends to deploy longer probe paths and thus generates more forwarding overhead with high telemetry latency thresholds.

6.2.3. The impact of the SAMLS algorithm on telemetry latency

This section discusses the impact of the SAMLS algorithm on telemetry latency. For fairness, all probe path planning algorithms are implemented within the INT-LLPP framework. It is important to note that the INT-LLPP framework is incompatible with the probe path planning algorithm in NetView, as it causes the confusion caused by split link-level elements. The design purpose of the SAMLS algorithm is to balance telemetry latency and control overhead. However, the importance of these two factors is different in specific network scenarios. To evaluate the performance of the SAMLS algorithm, we considered two representative network scenarios.

- Scenario 1: Consider networks such as online games and video conferences. Administrators require access to up-to-date telemetry information for network management. We call these network scenarios latency-aware networks and believe that telemetry latency is a more important metric. To visually demonstrate the importance of telemetry latency, we set the weight of telemetry latency ω_l to 0.9 and the weight of control overhead ω_c to 0.1.
- Scenario 2: For large-scale enterprise-level applications, administrators consider minimizing the control overhead of their network telemetry system to reduce computation and storage costs. We call these network scenarios overhead-aware networks. We believe that control overhead is a more important metric and set the weight of control overhead ω_c to 0.9 while the weight of telemetry latency ω_l is set to 0.1.

We first analyze the telemetry cost Q of the four methods in latency-aware networks. Fig. 12 shows the relationship between the telemetry cost Q and the number of switches in latency-aware networks. We find that the PPG algorithm with the maximum latency threshold determined by the SAMLS algorithm can achieve the best telemetry cost. This is because the SAMLS algorithm considers the importance of control overhead and telemetry latency, and selects a smaller maximum latency threshold in latency-aware networks. In contrast, other algorithms that use the fixed maximum latency threshold show similar performance, as the telemetry latency generated by these algorithms stabilizes around the fixed threshold after the number of switches reaches a certain scale. However, the selection of the maximum latency threshold

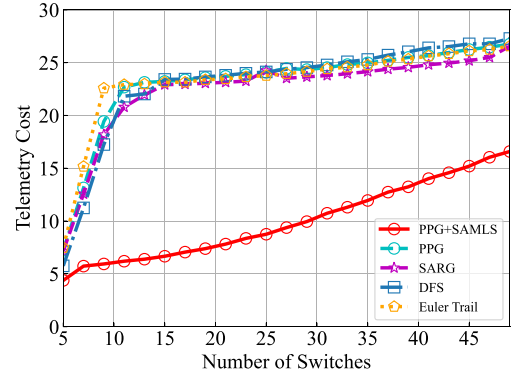


Fig. 12. Relationship between the telemetry cost and the number of switches in latency-aware networks.

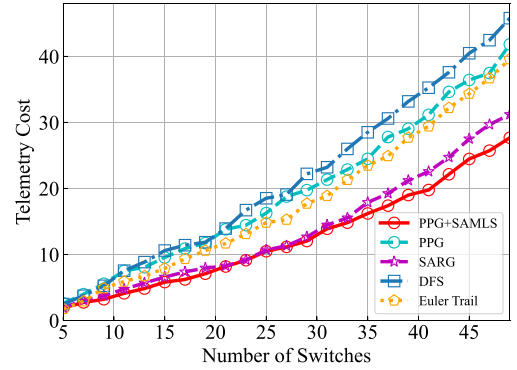


Fig. 13. Relationship between the telemetry cost and the number of switches in overhead-aware networks.

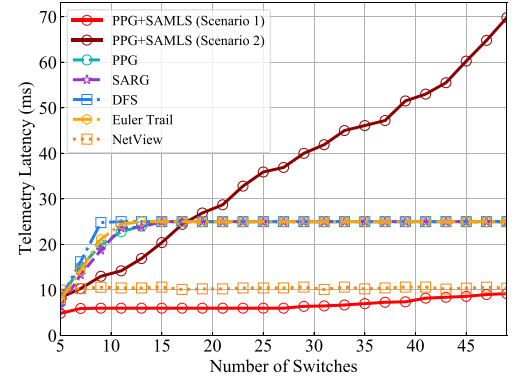


Fig. 14. Relationship between the telemetry latency and number of switches.

is subjective and cannot adapt to changes in the network topology scale. As a result, other algorithms that rely on fixed maximum latency thresholds may not achieve the ideal telemetry cost.

Next, we conducted a detailed analysis of the telemetry cost Q of the four methods in overhead-aware networks. As shown in Fig. 13, the method using the PPG algorithm and SAMLS algorithm achieves the best telemetry cost Q . Since control overhead is more important in overhead-aware networks, the SAMLS algorithm selects a larger latency threshold to reduce control overhead. It is worth noting that the performance of the SARG algorithm is also rated highly, particularly when the number of switches is between 20 and 30. This outcome suggests that the pre-set fixed latency threshold is close to the optimal latency threshold for this range of switch numbers, enabling the SARG algorithm to perform well under these conditions.

To further demonstrate the efficacy of the SAMLs algorithm, we analyzed the telemetry latency for each of the four methods tested in both latency-awared and overhead-awared networks. As shown in Fig. 14, the telemetry latency threshold set by the SAMLs algorithm varies depending on the specific telemetry cost and network environment. In latency-awared networks, where minimizing telemetry latency is essential to avoid deteriorating the telemetry cost Q , the SAMLs algorithm maintains a very low telemetry latency threshold. In contrast, in overhead-awared networks, where control overhead is a greater concern, the telemetry latency threshold set by the SAMLs algorithm increases as the network scale expands. This flexibility allows the SAMLs algorithm to dynamically adjust the telemetry latency threshold based on real-time conditions and the telemetry cost Q . By comparison, the fixed telemetry latency threshold approach used by the SARG algorithm, DFS algorithm, and Euler Trail algorithm results in a constant telemetry latency of 25 ms, regardless of scenario or network size. Notably, in overhead-awared networks, the telemetry latency threshold set by the SAMLs algorithm closely approximates the default telemetry latency threshold when the number of switches is 24, providing additional evidence to support the strong performance of the SARG algorithm in this scenario.

Overall, our findings underscore the importance of carefully selecting telemetry latency thresholds based on specific telemetry costs and environmental conditions. By using the SAMLs algorithm, AdapINT can achieve exceptional scalability and flexibility in dynamic network environments.

6.2.4. Convergence of the SAMLs algorithm

Achieving reliable convergence is critical when it comes to solving optimization problems, and the SAMLs algorithm based on simulated annealing also needs to consider this problem. In this algorithm, the temperature parameter and cooling rate both play essential roles in controlling the diversity and local search strength of the algorithm during the search process. To ensure optimal performance, it is crucial to select appropriate values for these parameters through experimentation and analysis. If the temperature decreases too quickly, the algorithm may become trapped in a suboptimal solution, while slow temperature reductions can result in longer computational times. Another factor that can impact the convergence of the SAMLs algorithm is the neighborhood structure used to generate new solutions at each temperature. By randomly generating $T_{\max}$ within a simple neighborhood structure, the algorithm ensures that it traverses the entire search space, maximizing its chances of finding the global optimal solution.

To evaluate the convergence performance of the SAMLs algorithm in our experiments, we recorded the value of the telemetry cost every twelve iterations. Fig. 15 illustrates the convergence behavior of the SAMLs algorithm for both latency-awared networks and overhead-awared networks. As shown in Fig. 15, the SAMLs algorithm reaches convergence after approximately 1000 iterations and finds optimal telemetry latency thresholds. These results demonstrate the effectiveness of the SAMLs algorithm in solving complex network telemetry problems.

7. Conclusions

In this paper, we propose INT-LLPP, a new in-band network-wide telemetry system that aims to reduce control overhead, forwarding overhead, and telemetry latency. To reduce control overhead, we designed a network telemetry architecture that allows each switch to add different types of telemetry items to the same probe. To reduce forwarding overhead, we propose the PPG algorithm to generate probe paths. Finally, to balance telemetry latency and control overhead, we propose the SAMLs algorithm to set the maximum latency threshold. Simulation results show that the telemetry architecture of INT-LLPP can significantly reduce the control overhead of network telemetry. The deployment scheme of the detection path using the PPG and SAMLs

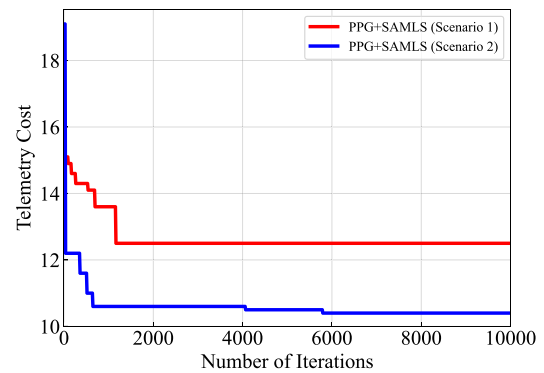


Fig. 15. Convergence of the SAMLs algorithm.

algorithms is effective in optimizing forwarding overhead, control overhead, and telemetry latency. Moreover, since the latency information is dynamic, the proposed path planning scheme should update the path deployment regularly. This work can be further extended by considering the impact of the frequency of redeploying paths on system performance and the issue of dynamic environments, which will be left as future work.

CRediT authorship contribution statement

Penghui Zhang: Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Methodology, Funding acquisition, Formal analysis, Data curation. **Hua Zhang:** Writing – review & editing. **Yuqi Dai:** Writing – review & editing. **Cheng Zeng:** Writing – review & editing. **Jingyu Wang:** Writing – review & editing. **Jianxin Liao:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] Y.-C. Wang, J. Xue, C. Wei, C.C.J. Kuo, An overview on generative AI at scale with edge-cloud computing, *IEEE Open J. Commun. Soc.* 4 (2023) 2952–2971.
- [2] M. Mohajeri, A. Ghassemi, T.A. Gulliver, Fast big data analytics for smart meter data, *IEEE Open J. Commun. Soc.* 1 (2020) 1864–1871.
- [3] S.K. Jagatheesaperumal, Q.-V. Pham, R. Ruby, Z. Yang, C. Xu, Z. Zhang, Explainable AI over the Internet of Things (IoT): Overview, state-of-the-art and future directions, *IEEE Open J. Commun. Soc.* 3 (2022) 2106–2136.
- [4] T. Yang, J. Jiang, P. Liu, Q. Huang, J. Gong, Y. Zhou, R. Miao, X. Li, S. Uhlig, Elastic sketch: adaptive and fast network-wide measurements, in: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 561–575, <http://dx.doi.org/10.1145/3230543.3230544>.
- [5] B. He, J. Wang, Q. Qi, H. Sun, J. Liao, C. Du, X. Yang, Z. Han, DeepCC: Multi-agent deep reinforcement learning congestion control for multi-path TCP based on self-attention, *IEEE Trans. Netw. Serv. Manag.* 18 (4) (2021) 4770–4788.
- [6] Y. Xie, F. Yi, K. Jamieson, PBE-CC: Congestion control via endpoint-centric, physical-layer bandwidth measurements, in: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 451–464, <http://dx.doi.org/10.1145/3387514.3405880>.

- [7] H. Wu, X. Zhang, T. Chen, G. Cheng, X. Hu, IM-shield: A novel defense system against DDoS attacks under IP spoofing in high-speed networks, in: ICC 2022 - IEEE International Conference on Communications, Seoul, Korea, Republic of, 2022, pp. 4168–4173, <http://dx.doi.org/10.1109/ICC45855.2022.9838621>.
- [8] A. Rezapour, W.-G. Tzeng, RL-shield: Mitigating target link-flooding attacks using SDN and deep reinforcement learning routing algorithm, *IEEE Trans. Dependable Secur. Comput.* 19 (6) (2022) 4052–4067.
- [9] V. Jeyakumar, M. Alizadeh, Y. Geng, C. Kim, D. Mazières, Millions of Little Minions: Using Packets for Low Latency Network Programming and Visibility, Vol. 44, Association for Computing Machinery, New York, NY, USA, 2014, pp. 3–14, <http://dx.doi.org/10.1145/2740070.2626292>.
- [10] B. Arzani, S. Ciraci, L. Chamon, Y. Zhu, H.H. Liu, J. Padhye, B.T. Loo, G. Outhred, 007: Democratically finding the cause of packet drops, in: 15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 18, USENIX Association, Renton, WA, 2018, pp. 419–435, URL: <https://www.usenix.org/conference/nsdi18/presentation/arzani>.
- [11] Y. Lin, Y. Zhou, Z. Liu, K. Liu, Y. Wang, M. Xu, J. Bi, Y. Liu, J. Wu, NetView: Towards on-demand network-wide telemetry in the data center, *Comput. Netw.* 180 (2020) 107386.
- [12] T. Barbette, C. Soldani, L. Mathy, Fast userspace packet processing, in: 2015 ACM/IEEE Symposium on Architectures for Networking and Communications Systems, ANCS, Oakland, CA, USA, 2015, pp. 5–16, <http://dx.doi.org/10.1109/ANCS.2015.7110116>.
- [13] H. Yao, T. Mai, X. Xu, P. Zhang, M. Li, Y. Liu, NetworkAI: An intelligent network architecture for self-learning control strategies in software defined networks, *IEEE Internet Things J.* 5 (6) (2018) 4319–4327.
- [14] R. Ben Basat, S. Ramanathan, Y. Li, G. Antichi, M. Yu, M. Mitzenmacher, PINT: Probabilistic in-band network telemetry, in: Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 662–680, <http://dx.doi.org/10.1145/3387514.3405894>.
- [15] C. Guo, L. Yuan, D. Xiang, Y. Dang, R. Huang, D. Maltz, Z. Liu, V. Wang, B. Pang, H.a. Chen, Pingmesh: A large-scale system for data center network latency measurement and analysis, *Comput. Commun. Rev.* 45 (4) (2015) 139–152.
- [16] Y. Li, R. Miao, H.H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, M. Yu, HPCC: high precision congestion control, in: Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 44–58, <http://dx.doi.org/10.1145/3341302.3342085>.
- [17] C. Kim, A. Sivaraman, N.P.K. Katta, A. Bas, A.A. Dixit, L.J. Wobker, In-band network telemetry via programmable dataplanes, 2015, URL: <https://api.semanticscholar.org/CorpusID:15782087>.
- [18] S. Tang, D. Li, B. Niu, J. Peng, Z. Zhu, Sel-INT: A runtime-programmable selective in-band network telemetry system, *IEEE Trans. Netw. Serv. Manag.* 17 (2) (2020) 708–721.
- [19] C.A. Sunshine, Source Routing in Computer Networks, Vol. 7, Association for Computing Machinery, New York, NY, USA, 1977, pp. 29–33, <http://dx.doi.org/10.1145/1024853.1024855>.
- [20] S. Tang, J. Kong, B. Niu, Z. Zhu, Programmable multilayer INT: An enabler for AI-assisted network automation, *IEEE Commun. Mag.* 58 (1) (2020) 26–32.
- [21] T. Pan, E. Song, C. Jia, W. Cao, T. Huang, B. Liu, Lightweight network-wide telemetry without explicitly using probe packets, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, Toronto, ON, Canada, 2020, pp. 1354–1355, <http://dx.doi.org/10.1109/INFOCOMWKSHPS50562.2020.9162684>.
- [22] Z. Liu, J. Bi, Y. Zhou, Y. Wang, Y. Lin, NetVision: Towards network telemetry as a service, in: 2018 IEEE 26th International Conference on Network Protocols, ICNP, Cambridge, UK, 2018, pp. 247–248, <http://dx.doi.org/10.1109/ICNP.2018.00036>.
- [23] T. Pan, E. Song, Z. Bian, X. Lin, X. Peng, J. Zhang, T. Huang, B. Liu, Y. Liu, INT-path: Towards optimal path planning for in-band network-wide telemetry, in: IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, Paris, France, 2019, pp. 487–495, <http://dx.doi.org/10.1109/INFOCOM.2019.8737529>.
- [24] D. Bhamare, A. Kessler, J. Vestin, M.A. Khoshkholghi, J. Taheri, IntOpt: In-band network telemetry optimization for NFV service chain monitoring, in: ICC 2019 - 2019 IEEE International Conference on Communications, ICC, Shanghai, China, 2019, pp. 1–7, <http://dx.doi.org/10.1109/ICC.2019.8761722>.
- [25] M. Yu, Network Telemetry: Towards a Top-Down Approach, Vol. 49, Association for Computing Machinery, New York, NY, USA, 2019, pp. 11–17, <http://dx.doi.org/10.1145/3314212.3314215>.
- [26] N. Katta, M. Hira, C. Kim, A. Sivaraman, J. Rexford, HULA: Scalable load balancing using programmable data planes, in: Proceedings of the Symposium on SDN Research, SOSR '16, Association for Computing Machinery, New York, NY, USA, 2016, <http://dx.doi.org/10.1145/2890955.2890968>.
- [27] S. Tang, S. Zhao, X. Pan, Z. Zhu, How to use in-band network telemetry wisely: Network-wise orchestration of sel-INT, *IEEE/ACM Trans. Netw.* 31 (1) (2023) 421–435.
- [28] Y. Zhu, N. Kang, J. Cao, A. Greenberg, G. Lu, R. Mahajan, D. Maltz, L. Yuan, M. Zhang, B.Y. Zhao, H. Zheng, Packet-Level Telemetry in Large Datacenter Networks, Vol. 45, Association for Computing Machinery, New York, NY, USA, 2015, pp. 479–491, <http://dx.doi.org/10.1145/2829988.2787483>.
- [29] T. Pan, X. Lin, H. Song, E. Song, Z. Bian, H. Li, J. Zhang, F. Li, T. Huang, C. Jia, B. Liu, INT-probe: Lightweight in-band network-wide telemetry with stationary probes, in: 2021 IEEE 41st International Conference on Distributed Computing Systems, ICDCS, DC, USA, 2021, pp. 898–909, <http://dx.doi.org/10.1109/ICDCS51616.2021.00090>.
- [30] A. Arins, Latency factor in worldwide IP routed networks, in: 2014 IEEE 2nd Workshop on Advances in Information, Electronic and Electrical Engineering, AIEEE, Vilnius, Lithuania, 2014, pp. 1–4, <http://dx.doi.org/10.1109/AIEEE.2014.7020316>.
- [31] A.V. Aho, J.E. Hopcroft, The Design and Analysis of Computer Algorithms, first ed., Addison-Wesley Longman Publishing Co., Inc., USA, 1974.
- [32] Y. Lin, Y. Zhou, Z. Liu, K. Liu, Y. Wang, M. Xu, J. Bi, Y. Liu, J. Wu, Netview: Towards on-demand network-wide telemetry in the data center, *Comput. Netw.* 180 (2020) 107386.
- [33] A. Roy, K. Yocum, A.C. Snoeren, Challenges in the emulation of large scale software defined networks, in: Proceedings of the 4th Asia-Pacific Workshop on Systems, APSys '13, Association for Computing Machinery, New York, NY, USA, 2013, <http://dx.doi.org/10.1145/2500727.2500738>.
- [34] A. El-mekkawi, X. Hesselbach, J.R. Piney, Evaluating the impact of delay constraints in network services for intelligent network slicing based on SKM model, *J. Commun. Netw.* 23 (4) (2021) 281–298.